

BİNARY SEARCH TREE (BST)

Yapı · Arama · Ekleme · Dolaşım · Level-Order · Karmaşıklık Analizi

BST Özelliği: Her düğüm v için: sol alt ağacındaki tüm anahtarlar $< v.key$ ve sağ alt ağacındaki tüm anahtarlar $> v.key$

1. Veri Yapısı – Node Tanımı

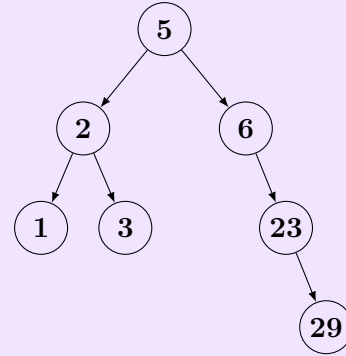
BST Yapısı – Node Tanımı

Java Kodu

```
private class Node {
    public int key;
    public Node left;
    public Node right;

    public Node(int key) {
        this.key = key;
        left = right = null;
    }
}
```

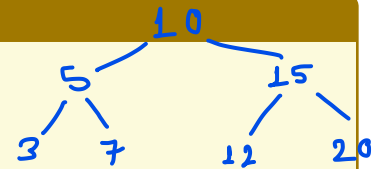
Örnek BST: Ekleme sırası
{5,2,6,1,3,23,29}



Özellik	Açıklama
Kök (root)	Ağacın en üst düğümü (ebeveyn = null)
Yaprak (leaf)	Sol ve sağ çocukları null olan düğüm
Yükseklik (h)	Kökten en uzak yaprağa olan yol uzunluğu
Denge	Dengeli BST: $h = O(\log n)$; dengesiz: $h = O(n)$

Pratik – BST Yapısı

(a) {10, 5, 15, 3, 7, 12, 20} sırasıyla eklendiğinde ağacı çizin.



(b) {1, 2, 3, 4, 5} eklendiğinde yükseklik nedir? Bu hangi duruma karşılık gelir? 5
 $O(n)$

2. Arama – Rekürsif Search

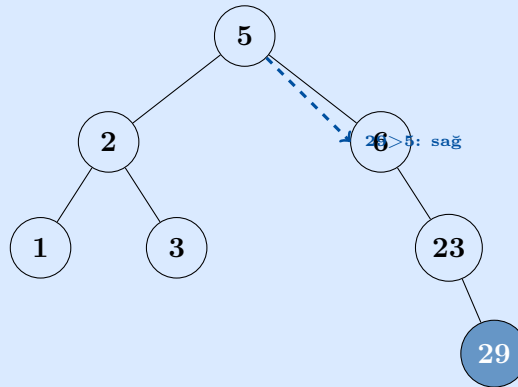
Arama (Search)

Rekürsif Kod

```
Node search(Node root, int key) {
    if (root == null || root.key == key)
        return root;
    if (key < root.key)
        return search(root.left, key);
    return search(root.right, key);
}
```

```
// Çağrı:
search(root, 29);
```

Arama Görseli: `search(root, 29)`



Yol: $5 \rightarrow 6 \rightarrow 23 \rightarrow 29$ (3 adım)

Karmaşıklık

Durum	Açıklama	Karmaşıklık
En iyi	Kökte bulunur	$O(1)$
Ortalama	Dengeli BST	$O(\log n)$
En kötü	Lineer (dengesiz)	$O(n)$

Pratik – Arama

(a) Yukarıdaki ağaçta `search(root, 4)` kaç adım atar? Yolu yazın. 4 $\{5 \rightarrow 2 \rightarrow 3 \rightarrow \text{null}\}$ (b)
`search(root, 100)` nasıl sonlanır? 5 → 6 → 23 → 29 → null bulunamadı.

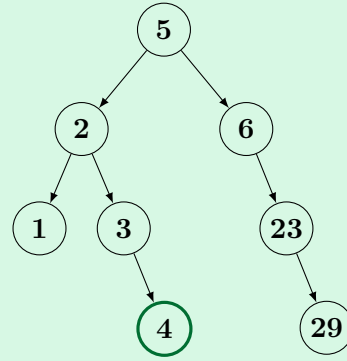
3. Ekleme – Iteratif Insert

Ekleme (Iteratif Insert)

Kod

```
Node insert(Node root, int key) {
    Node newNode = new Node(key);
    if (root == null) return newNode;
    Node current = root;
    Node parent = null;
    while (current != null) {
        parent = current;
        if (key < current.key)
            current = current.left;
        else
            current = current.right;
    }
    if (key < parent.key)
        parent.left = newNode;
    else
        parent.right = newNode;
    return root;
}
```

Ekleme Adımları: 4 ekleniyor

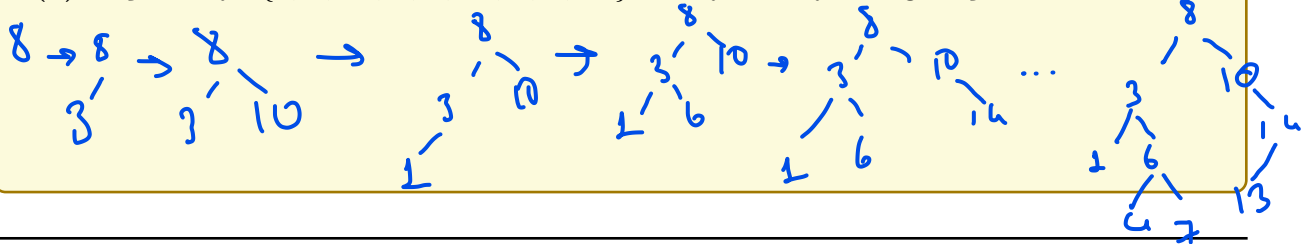


$5 \xrightarrow{4 < 5} 2 \xrightarrow{4 > 2} 3 \xrightarrow{4 > 3}$ sağına ekle

	En Kötü	Ortalama
Ekleme	$O(n)$ (lineer ağaç)	$O(\log n)$
Silme	$O(n)$	$O(\log n)$
Arama	$O(n)$	$O(\log n)$
Güncelleme	$O(n)$	$O(\log n)$

Pratik – Ekleme

(a) Boş BST'ye {8, 3, 10, 1, 6, 14, 4, 7, 13} sırasıyla ekleyin. Ağacı çizin.



4. Dolaşım (Traversal)

Dolaşım (Traversal)

Inorder (Sol-Kök-Sağ)

```
void inorder(Node root) {
    if (root != null) {
        inorder(root.left);
        print(root.key);
        inorder(root.right);
    }
}
```

Sonuç: Sıralı (*artan*)
1, 2, 3, 5, 6, 23, 29

Preorder (Kök-Sol-Sağ)

```
void preorder(Node root) {
    if (root != null) {
        print(root.key);
        preorder(root.left);
        preorder(root.right);
    }
}
```

Sonuç: Kök önce
5, 2, 1, 3, 6, 23, 29

Postorder (Sol-Sağ-Kök)

```
void postorder(Node root) {
    if (root != null) {
        postorder(root.left);
        postorder(root.right);
        print(root.key);
    }
}
```

Sonuç: Kök en son
1, 3, 2, 29, 23, 6, 5

Level-Order (BFS – Kuyrukla)

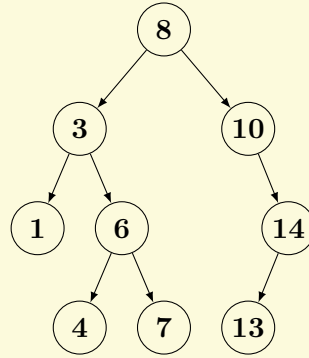
```
void levelOrder(Node root) {
    if (root == null) return;
    Queue<Node> q = new Queue<>();
    q.enqueue(root);
    while (!q.isEmpty()) {
        Node current = q.dequeue();
        System.out.println(current.key);
        if (current.left != null) q.enqueue(current.left);
        if (current.right != null) q.enqueue(current.right);
    }
}
```

Sonuç (seviye seviye): 5 | 2, 6 | 1, 3, 23 | 29

Dolaşım	Sıra	Kullanım	Karmaşıklık
Inorder	Sol-Kök-Sağ	Sıralı liste	$\Theta(n)$
Preorder	Kök-Sol-Sağ	Kopyalama, seri	$\Theta(n)$
Postorder	Sol-Sağ-Kök	Silme, ifade	$\Theta(n)$
Level-Order	Seviye seviye	BFS, kısa yol	$\Theta(n)$

Pratik – Dolaşım

Aşağıdaki ağaç için dört dolaşımın çıktısını yazın:



Inorder: _____ Preorder: _____ Postorder: _____ Level-Order: _____
 [1 3 4 6 7 8 10 13 14] [8 3 1 6 4 7 10 14 13] [1 4 7 6 3 13 14 10 8] [8 3 10 1 6 14 4 7 13]

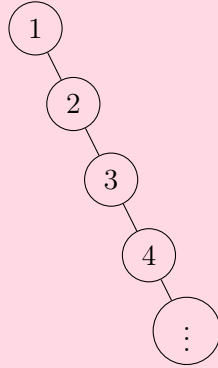
5. Karmaşıklık Analizi

Karmaşıklık Analizi

5.1 – En Kötü Durum: Lineer Ağaç

Anahtarlar sıralı eklendiğinde $(1, 2, 3, \dots, n)$ ağaç sağa eğilir. Her ekleme/arama bir öncekinin sağ alt ağacına gider.

$$T(n) = T(n-1) + c, \quad T(1) = c \implies T(n) = cn = O(n)$$



$$h = n - 1 \implies T(n) = O(n)$$

5.2 – Ortalama Durum: Dengeli Ağaç

Rastgele ekleme sırasında ağaç dengeli kalır: $T(n) = T(n/2) + c$

$$T(n) = T(n/2) + c \xrightarrow{\text{Master}} f(n) = n^0, \quad E = \log_2 1 = 0, \quad c = 0 = E \implies T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

5.3 – BST Ortalama Maliyet Türetmesi

Rastgele n elemanın eklendiği BST’de bir arama kaç düğüm ziyaret eder? Bu, “ortalama başarılı arama maliyeti”nin türevidir.

$T(n)$: Tüm düğümlerin toplam derinliği (internal path length)

Her seviyede düğüm sayısı:

$$T(n) = \underbrace{2^0 \cdot \log 2^0}_{\text{sev.0}} + \underbrace{2^1 \cdot \log 2^1}_{\text{sev.1}} + \cdots + \underbrace{2^{\log n} \cdot \log 2^{\log n}}_{\text{sev. log } n} = \sum_{i=0}^{\log n} 2^i \cdot i$$

Toplam hesabı: $S = \sum_{i=0}^{\log n} i \cdot 2^i$

Bilinen sonuç: $\sum_{i=0}^m i \cdot 2^i = (m-1) \cdot 2^{m+1} + 2$

$m = \log n$ koyarak:

$$S = (\log n - 1) \cdot 2^{\log n + 1} + 2 = (\log n - 1) \cdot 2n + 2 = 2n \log n - 2n + 2$$

Ortalama arama maliyeti (eleman başına):

$$\frac{T(n)}{n} = \frac{2n \log n - 2n + 2}{n} = 2 \log n - 2 + \frac{2}{n} \approx \log n - 1 + \frac{1}{n}$$

$$T(n) = \Theta(n \log n) \text{ (toplam)}, \quad \frac{T(n)}{n} = \Theta(\log n) \text{ (ortalama arama)}$$

5.4 – Özet: Tüm Operasyonlar

Operasyon	En İyi	Ortalama	En Kötü	Alan
Arama	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$
Ekleme	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$
Silme	$O(1)$	$O(\log n)$	$O(n)$	$O(h)$
Inorder	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(h)$
Level-Order	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$

h = ağaç yüksekliği: dengeli ise $O(\log n)$, dengesiz ise $O(n)$

Pratik – Karmaşıklık

(a) $T(n) = T(n-1) + c$, $T(1) = c$ bağlantısını aç $T(n) = cn$ olduğunu gösterin. _____

$$\begin{aligned} T(n-1) &= T(n-2) + c \\ T(n) &= T(n-1) + c \\ &\vdots \\ T(n) &= T(1) + (n-1)c \\ &= c + (n-1)c = nc \end{aligned}$$

(b) $T(n) = T(n/2) + c$ için Master Teorem adımlarını yazın. _____

$$\begin{aligned} a &= 1 \\ b &= 2 \\ f(n) &= 1 \\ n^{\log_2 b} &= f(n) \\ 1 &= 1 \rightarrow O(\log n) \end{aligned}$$

(c) Dengeli BST ile dengesiz BST'yi karşılaştırın. $n = 1000$ için fark nedir? _____

$O(\log n)$

≈ 10

$O(n)$

$= 1000$

6. BST vs Diğer Veri Yapıları

Yapı	Arama	Ekleme	Silme	Sıralı?
Dizi (sırasız)	$O(n)$	$O(1)$	$O(n)$	Hayır
Dizi (sıralı)	$O(\log n)$	$O(n)$	$O(n)$	Evet
Bağlı liste	$O(n)$	$O(1)$	$O(n)$	Hayır
Hash tablosu	$O(1)^*$	$O(1)^*$	$O(1)^*$	Hayır
BST (ort.)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Evet
AVL / R-B Ağ.	$O(\log n)$	$O(\log n)$	$O(\log n)$	Evet

* Ortalama; çakışma durumunda $O(n)$

BST Ne Zaman Tercih Edilir?

- Sıralı erişim gerekiyorsa (inorder $\Rightarrow$ sıralı çıktı)
- Dinamik ekleme/silme + hızlı arama birlikte gerekiyorsa
- k . en küçük/en büyük sorgusu gerekiyorsa
- **Dikkat:** Dengeli BST (AVL, Red-Black) garantili $O(\log n)$; sade BST $O(n)$ olabilir

Pratik – Karma Sorular

(a) Inorder dolaşım neden sıralı çıktı üretir? BST özelliğiyle ilişkilendirin. _____

önce en sola inip daha sonrasında root ve sonra sağdakilere baktığı için sıralı çıktı verir.

(b) Level-Order neden DFS değil BFS kullanır? Kuyruk yerine yığın kullansaydık ne olurdu? _____

(c) n elemanın tamamını BST'ye sıralı eklersek (1,2,...,n), ortalama arama maliyeti nedir? $O(n)$

(d) Aynı elemanları rastgele sırayla eklersek beklenen ortalama arama maliyeti nedir?

$O(\log n)$